

XML PROGRAMMING (complete book, chp-12*short version)

27 Eylül 2016 Salı 10:54

XPATH (describe set of similar elements)
XQUERY (allow iterations over sets, subqueries.)
XSLT (transformation language)

XML → XSLT → HTML
 } XML result.

≡ XQuery expressive power.

XPATH:

SRL produces BKG of tuples.

XPATH " sequence of items.

value of primitive type

node: - Document : doc("movies.xml")
 - Element : /.../
 - Attribute : /.../@

Example of sequence of items: 10 "ten" 100 <Number base="8"> @val="10":
 <Digit> 1 </Digit>
 <Digit> 2 </Digit>
 </Number>

/T1/T2.../ ⇒ T1 is root tag

T1 is sequence of 1 item = representing the text in the file.

```
1) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
2) <StarMovieData>
3)   <Star starID = "cf" starredIn = "sw">
4)     <Name>Carrie Fisher</Name>
5)     <Address>
6)       <Street>123 Maple St.</Street>
7)       <City>Hollywood</City>
8)     </Address>
9)     <Address>
10)      <Street>5 Locust Ln.</Street>
11)      <City>Malibu</City>
12)    </Address>
13)   </Star>
14)   <Star starID = "mh" starredIn = "sw">
15)     <Name>Mark Hamill</Name>
16)     <Street>456 Oak Rd.</Street>
17)     <City>Brentwood</City>
18)   </Star>
19)   <Movie movieID = "sw" starOf = "cf", "mh">
20)     <Title>Star Wars</Title>
21)     <Year>1977</Year>
22)   </Movie>
23) </StarMovieData>
```

/StarMovieData

/StarMovieData/Star

/StarMovieData/Star/Name

result sequence is: <Name> Carrie...</Name>
 <Name> Mark...</Name>

Figure 12.2: An XML document for applying path expressions

/StarMovieData/Star/@starID

result sequence is: "cf", "mh"

AXES:

axis name	shorthand
child::	/
attribute::	@
parent::	..
ancestor::	]
descendant::	
next-sibling::	
self::	.

descendant-or-self // → lists sequence of elements at any level of nesting.

```

1) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
2) <StarMovieData>
3)   <Star starID = "cf" starredIn = "sw">
4)     <Name>Carrie Fisher</Name>
5)     <Address>
6)       <Street>123 Maple St.</Street>
7)       <City>Hollywood</City>
8)     </Address>
9)   </Star>
10)  <Star starID = "mh" starredIn = "sw">
11)    <Name>Mark Hamill</Name>
12)    <Street>456 Oak Rd.</Street>
13)    <City>Brentwood</City>
14)  </Star>
15)  <Movie movieID = "sw" starsOf = "cf", "mh">
16)    <Title>Star Wars</Title>
17)    <Year>1977</Year>
18)  </Movie>
19) </StarMovieData>

```

Figure 12.2: An XML document for applying path expressions

Find all cities where stars live?

at any level of nesting under root.
 or
 /StarMovieData/Star//City
 at any level of nesting under Star.

WILDCARDS:

* : any tag
 @* : any attribute

/StarMovieData/* ⇒ 2 stars and 1 movie
 /StarMovieData/@* ⇒ 'cf' 'sw' ... 'sw' 'cf' 'mh'

CONDITIONS:

/.tag/tag[condition]/.tag

= >=
 or !=

```

1) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
2) <StarMovieData>
3)   <Star starID = "cf" starredIn = "sw">
4)     <Name>Carrie Fisher</Name>
5)     <Address>
6)       <Street>123 Maple St.</Street>
7)       <City>Hollywood</City>
8)     </Address>
9)   </Star>
10)  <Star starID = "mh" starredIn = "sw">
11)    <Name>Mark Hamill</Name>
12)    <Street>456 Oak Rd.</Street>
13)    <City>Brentwood</City>
14)  </Star>
15)  <Movie movieID = "sw" starsOf = "cf", "mh">
16)    <Title>Star Wars</Title>
17)    <Year>1977</Year>
18)  </Movie>
19) </StarMovieData>

```

Figure 12.2: An XML document for applying path expressions

Name of stars who have at least 1 home in Malibu:

/StarMovieData/Star[City = "Malibu"]/Name

City element at only level of nesting.

other conditions.

[i] takes ith child
 [T] takes elements that have >1 subelements w/ tag T.
 [A] " " w/ att. A.

```

1) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
2) <Movies>
3)   <Movie title = "King Kong">
4)     <Version year = "1933">
5)       <Star>Fay Wray</Star>
6)     </Version>
7)   <Movie title = "The Godfather">
8)     <Version year = "1976">
9)       <Star>Al Pacino</Star>
10)    </Version>
11)  </Movies>

```

List year of first version of each movie.

/Movies/Movie/Version[i]/@year → 1933
 → 1984

```

8)      <Star>Jeff Bridges</Star> :
9)      <Star>Jessica Lange</Star>
10)     </Version>
11)     <Version year = "2005" />
12)  </Movie>
13)  <Movie title = "Footloose">
14)    <Version year = "1984">
15)      <Star>Kevin Bacon</Star>
16)      <Star>John Lithgow</Star>
17)      <Star>Sarah Jessica Parker</Star>
18)    </Version>
19)  </Movie>
20) </Movies>

```

Figure 12.3: An XML document for applying path expressions

version of movie

/Movies/Movie/Version[i]/@year → 1933
→ 1984

*List of versions that has
a newcoming star.*

/Movies/Movie/Version[Star] ~ /@year

List all versions except line 11.

*1933)
1984)
1985)*

XQUERY: $\Rightarrow$ xpath. , $XPath + FLWR^0 = XQuery$

- standard for querying SD in XML form.
- case-sensitive language.
- produces sequence of items:
 - ↳ primitive
 - ↳ nodes of different types
 - elements
 - attributes
 - documents.
- a functional language. "XQuery expression."

$\{ \text{for clause} \mid \text{let clause} \}^*$	- variables in "let clause" start w/ \$	
$\{ \text{where cond.} \}^+$		
return exp.		- for variable in expression

- Ex: `return <Greeting> hello </Greeting>`
- Ex: `let $stars := doc("stars.xml")`
- Ex: `let $movies := doc("movies.xml")`
`for $m in $movies/Movies/Movie`

execute here
for each "Movie"

```

1) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
2) <Stars>
3)   <Star>
4)     <Name>Carrie Fisher</Name>
5)     <Address>
6)       <Street>123 Maple St.</Street>
7)       <City>Hollywood</City>
8)     </Address>
9)   </Star>
10)  <Star>
11)    <Name>5 Locust Ln.</Name>
12)    <Address>
13)      <Street>5 Locust Ln.</Street>
14)      <City>Malibu</City>
15)    </Address>
16)  </Star>
17)  ... more stars
18) </Stars>

```

(a) Document stars.xml

```

15) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
16) <Movies>
17)   <Movie title = "King Kong">
18)     <Version year = "1933">
19)       <Star>Fay Wray</Star>
20)     </Version>
21)   </Movie>
22)   <Movie title = "1976">
23)     <Star>Jeff Bridges</Star>
24)     <Star>Jessica Lange</Star>
25)   </Movie>
26)   <Movie title = "2005" />
27) </Movies>
28) <Movie title = "Footloose">
29)   <Version year = "1984">
30)     <Star>Kevin Bacon</Star>
31)     <Star>John Lithgow</Star>
32)     <Star>Sarah Jessica Parker</Star>
33)   </Version>
34) </Movie>
35) ... More movies
36) </Movies>

```

(b) Document movies.xml

- list of all star elements among the versions of movies.

```

let $movies := doc('movies.xml')
for $m in $movies/Movies/Star
return $m/Version/Star

```

- list sequence of Movies including their own stars, regardless of which version they starred in.

```

let $movies := doc('movies.xml')
for $m in $movies/Movies/Star
return <Movie title = { $m/@title }> { $m/Version/Star } </Movie>

```

{ <Movie title = ...>
 {Star} ... {Star}
 ...
 { </Movie>
 { <Movie title = ...>
 ...
 { </Movie>

in order to get text interpreted as XQuery exp. inside tags; surround text by curly brackets. { }

- list all stars b/w <Stars>...</Stars> tag.

```

let $starSeq := (
  let $movies := doc('movies.xml')
  for $m in $movies/Movie
  return $m/Version/Star
)
return <Stars> { $starSeq } </Stars>

```

JOIN in XQUERY:

Join condition in where clause.

mostly compare data in elements. data() extracts value from element.

- list cities where stars live.

```

let $movies := doc('movies.xml')
let $stars := doc('stars.xml')

for $s1 in $movies/Movies/Star/Version/Star,
    $s2 in $stars/Star
where data($s1) = data($s2/Name)
return $s2/Address/City.

```

COMPARISON OPERATORS:

=	eq
>	gt
<	lt
↓	
compare sequences w/ a primitive true if one element equals.	compare sequences consisting of a single element. if sequence w/ .. "n" ..

equation.

many elements returns false.

```

1) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
2) <Stars>
3)   <Star>
4)     <Name>Carrie Fisher</Name>
5)     <Address>
6)       <Street>123 Maple St.</Street>
7)       <City>Hollywood</City>
8)     </Address>
9)   </Star>
10)  <Star>
11)    <Name>Jeff Bridges</Name>
12)    <Address>
13)      <Street>5 Locust Ln.</Street>
14)      <City>Malibu</City>
15)    </Address>
16)  </Star>
17)  ... more stars
18) </Stars>

```

(a) Document stars.xml

```

15) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
16) <Movies>
17)   <Movie title = "King Kong">
18)     <Version year = "1933">
19)       <Star>Fay Wray</Star>
20)     </Version>
21)   </Movie>
22)   <Movie title = "1976">
23)     <Star>Jeff Bridges</Star>
24)     <Star>Jessica Lange</Star>
25)   </Movie>
26)   <Movie title = "2005" />
27)   <Movie title = "Footloose">
28)     <Version year = "1984">
29)       <Star>Kevin Bacon</Star>
30)       <Star>John Lithgow</Star>
31)       <Star>Sarah Jessica Parker</Star>
32)     </Version>
33)   </Movie>
34)   ... more movies
35) </Movies>

```

(b) Document movies.xml

- Find who lives at 123 Maple St., Malibu.

let \$stars := doc('stars.xml')

for \$s in \$stars/Star

where \$s/Address/Street = "123 ..." and
\$s/Address/City = "Malibu"

return \$s/Name.

⇒ returns "Carrie Fisher". But wrong result.

⇒ returns nothing. Because left-side is sequence.

where \$s/Address[data(Street) eq "123..."] and
\$s/Address[data(City) eq "Malibu"]

- DUPLICATE ELIMINATION: distinct-values(...). → outputs list of strings.
seq. of elements

let \$x := distinct-values(<A>1
<A>2
<A>1)

return <As> { \$x } </As> → outputs 1 2

- QUANTIFICATION (for all, there exist)

every var in exp1 satisfies exp2 → true/false.
(some) involves 'var'

- Find stars who only live in Hollywood.

let \$stars := doc('stars.xml')

for \$s in \$stars/Stars/Star

where every \$c in \$s/Address/City satisfies \$c = "Hollywood"

return \$s/Name

- Find stars who has a home in 'Hollywood'. (do not need some var in exp...)

let \$stars := doc('stars.xml')

for \$s in \$stars/Stars/Star

where \$s/Address/City = 'Hollywood'

return \$s/Name

- AGGREGATIONS:

- Find movies w/ multiple versions.

```
let $movies :=  
for $m in $movies/Movies/Movie  
where count($m/Version) > 1  
return $m
```

- BRANCHING:

if (exp1). then exp2 else exp3

you cannot omit the part.
use else ()

- List versions of 'King Kong' movie with tagging
the most recent version 'Latest' and earlier versions 'old'.

```
let $kk := doc("movies.xml")/Movies/Movie[@title = 'kk']  
for $v in $kk/Version  
return  
if ($v/@year = max($kk/Version/@year))  
then <Latest> { $v } </Latest>  
else <Old> { $v } </Old>
```

- ORDERING:

order
return

- List seq. of Movie elements ordered by year w/ title & year
as their attributes.

```
1) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>  
2) <Stars>  
3)   <Star>  
4)     <Name>Carrie Fisher</Name>  
5)     <Address>  
6)       <Street>123 Maple St.</Street>  
7)       <City>Hollywood</City>  
8)     </Address>  
9)   </Star>  
10)  <Star>  
11)    <Street>5 Locust Ln.</Street>  
12)    <City>Malibu</City>  
13)  </Star>  
14)  ... More stars  
15) </Stars>
```

(a) Document stars.xml

```
15) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>  
16) <Movies>  
17)   <Movie title = "King Kong">  
18)     <Version year = "1933">  
19)       <Star>Fay Wray</Star>  
20)     </Version>  
21)     <Version year = "1976">  
22)       <Star>Jeff Bridges</Star>  
23)       <Star>Jessica Lange</Star>  
24)     </Version>  
25)     <Version year = "2005" />  
26)   </Movie>  
27)   <Movie title = "Footloose">  
28)     <Version year = "1984">  
29)       <Star>Kevin Bacon</Star>  
30)       <Star>John Lithgow</Star>  
31)       <Star>Sarah Jessica Parker</Star>  
32)     </Version>  
33)   </Movie>  
34)   ... More movies  
35) </Movies>
```

(b) Document movies.xml

```
let $movies := doc("movies.xml")
```

```
for $m in $movies/Movies/Movie  
  $v in $m/Version
```

ordered by \$v/@year, \$m/@title → breaks ties.

```
return <Movie title = {$m/@title} year = {$v/@year}>
```

```

<Products>
  <Maker name = "A">
    <PC model = "1001" price = "2114">
      <Speed>2.66</Speed>
      <RAM>1024</RAM>
      <HardDisk>250</HardDisk>
    </PC>
    <PC model = "1002" price = "995">
      <Speed>2.10</Speed>
      <RAM>512</RAM>
      <HardDisk>250</HardDisk>
    </PC>
    <Laptop model = "2004" price = "1150">
      <Speed>2.00</Speed>
      <RAM>512</RAM>
      <HardDisk>60</HardDisk>
      <Screen>13.3</Screen>
    </Laptop>
    <Laptop model = "2005" price = "2500">
      <Speed>2.16</Speed>
      <RAM>1024</RAM>
      <HardDisk>120</HardDisk>
      <Screen>17.0</Screen>
    </Laptop>
  </Maker>

```

```

<Maker name = "E">
  <PC model = "1011" price = "959">
    <Speed>1.86</Speed>
    <RAM>2048</RAM>
    <HardDisk>160</HardDisk>
  </PC>
  <PC model = "1012" price = "649">
    <Speed>2.80</Speed>
    <RAM>1024</RAM>
    <HardDisk>160</HardDisk>
  </PC>
  <Laptop model = "2001" price = "3673">
    <Speed>2.00</Speed>
    <RAM>2048</RAM>
    <HardDisk>240</HardDisk>
    <Screen>20.1</Screen>
  </Laptop>
  <Printer model = "3002" price = "239">
    <Color>false</Color>
    <Type>laser</Type>
  </Printer>
</Maker>
<Maker name = "H">
  <Printer model = "3006" price = "100">
    <Color>true</Color>
    <Type>ink-jet</Type>
  </Printer>
  <Printer model = "3007" price = "200">
    <Color>true</Color>
    <Type>laser</Type>
  </Printer>
</Maker>
</Products>

```

Exercise 12.2.1: Using the product data from Figs. 12.4 and 12.5, write the following in XQuery.

- Find the Printer elements with a price less than 100.
- Find the Printer elements with a price less than 100, and produce the sequence of these elements surrounded by a tag <CheapPrinters>.
- Find the names of the makers of both printers and laptops.
- Find the names of the makers that produce at least two PC's with a speed of 3.00 or more.
- Find the makers such that every PC they produce has a price no more than 1000.
- Produce a sequence of elements of the form

```
<Laptop><Model>x</Model><Maker>y</Maker></Laptop>
```

where x is the model number and y is the name of the maker of the laptop.

A)

```

let $products := doc(" ")
for $p in $products/maker/printer
where $p/@price < 100
return $p

```

→ \$products//printer

→ lt

B)

```

let $cheapPrinters := (
  (A)
)
return <cheapPrinters> { $cheapPrinters } </cheapPrinters>

```

C)

```

let $products := doc("../xml")
for $m in $products/maker
where $m/printer and $m/laptop
return $m/@name

```

returns a seq.
if empty seq => false
is \$t exists => true.

→ <maker name = { \$m/@name } />

```

<Ships>
  <Class name = "Kongo" type = "bc" country = "Japan"
    numGuns = "8" bore = "14" displacement = "32000">
    <Ship name = "Kongo" launched = "1913" />
    <Ship name = "Hiei" launched = "1914" />
    <Ship name = "Kirishima" launched = "1915">
      <Battle outcome = "sunk">Guadalcanal</Battle>
    </Ship>
  <Class name = "Haruna" launched = "1915" />
</Class>
<Class name = "North Carolina" type = "bb" country = "USA"
  numGuns = "9" bore = "16" displacement = "37000">
  <Ship name = "North Carolina" launched = "1941" />
  <Ship name = "Washington" launched = "1941">
    <Battle outcome = "ok">Guadalcanal</Battle>
  </Ship>
</Class>
<Class name = "Tennessee" type = "bb" country = "USA"
  numGuns = "12" bore = "14" displacement = "32000">
  <Ship name = "Tennessee" launched = "1920">
    <Battle outcome = "ok">Surigao Strait</Battle>
  </Ship>
  <Ship name = "California" launched = "1921">
    <Battle outcome = "ok">Surigao Strait</Battle>
  </Ship>
</Class>
<Class name = "King George V" type = "bb"
  country = "Great Britain"
  numGuns = "10" bore = "14" displacement = "32000">
  <Ship name = "King George V" launched = "1940" />
  <Ship name = "Prince of Wales" launched = "1941">
    <Battle outcome = "damaged">Denmark Strait</Battle>
    <Battle outcome = "sunk">Malaya</Battle>
  </Ship>
  <Ship name = "Duke of York" launched = "1941">
    <Battle outcome = "ok">North Cape</Battle>
  </Ship>
  <Ship name = "Hove" launched = "1942" />
  <Ship name = "Anson" launched = "1942" />
</Class>
</Ships>

```

Exercise 12.2.2: Using the battleships data of Fig. 12.6, write the following in XQuery.

- Find the names of the classes that had at least 10 guns.
- Find the names of the ships that had at least 10 guns.
- Find the names of the ships that were sunk.
- Find the names of the classes with at least 3 ships.
- Find the names of the classes such that no ship of that class was in a battle.
- Find the names of the classes that had at least two ships launched in the same year.
- Produce a sequence of items of the form

<Battle name = x><Ship name = y />...</Battle>

there may be many ships in a battle.

A)

```

let $ships := doc("...")
for $c in $ships//Class
where $c/@numGuns > 10
return ($c/@name)

```

data (also da olur, olmaz da olur)

```

let $ships := doc("...")
for $c in $ships//Class
where $c/@numGuns > 10
return ($c/Ship/@name)

```

B)

data